MESSMAP

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1. Purpose and System Description

This SRS defines the software requirements for implementing MessMap. It is intended for developers, testers, project guides, and team members. It turns the business goals and functional behavior into requirements that can guide architecture, coding, and testing.

2. Proposed Technology Architecture

Layer	Technology	Responsibility
Frontend	React.js	Pages, forms, dashboards, search results, filters, validation feedback, and map display.
Backend	Java Spring Boot	REST APIs, business rules, authentication/authorization, verification workflow, search logic, and data validation.
Database	MySQL	Persistent storage for accounts, roles, mess listings, menus, prices, availability, and reviews.
Map/location service	Provider to be selected	Location lookup, map display, distance/directions support, subject to provider capabilities.

3. User Classes and Access Control

Role	Permitted access
Student	Public approved mess listings; search and filters; details; reviews and ratings; own account functions.
Mess Owner	Owner dashboard; create and manage the listing associated with that owner; view verification status.
Admin	Administrative dashboard; user/owner/listing management; listing verification; review moderation.

The backend must enforce permissions. Hiding a button in React is not sufficient security; Spring Boot must check the user's identity and role for protected operations.

4. Functional System Requirements

The system shall expose backend REST APIs for registration, login, mess discovery, mess details, owner updates, listing verification, and review management.

The backend shall validate incoming request data before writing it to the database.

The system shall store a status for each mess listing, at minimum Pending, Approved, and Rejected; an Active/Inactive status may be used separately for operating status.

Search results shall include only listings approved for public display and not administratively disabled.

The system shall support location-based search using coordinates or a selected address, depending on the chosen map/location integration.

The system shall return clear success/error responses that the frontend can display.

Administrative changes and listing-status changes should be traceable through timestamps and, where practical, an audit log.

5. Data Requirements

The following is a suggested initial data model. Field names and normalization can be adjusted during database design.

Entity / Table	Suggested key fields	Purpose
users	user_id, name, email, password_hash, phone, role, status, created_at	Stores account identity, role, and account status.
mess_listings	mess_id, owner_id, name, description, address, latitude, longitude, contact, food_type, verification_status, operating_status, created_at, updated_at	Stores each mess profile and verification/operating status.
menus	menu_id, mess_id, day/date, meal_type, menu_description, availability	Stores daily or weekly menu details.
pricing	price_id, mess_id, plan_type, meal_type, amount, valid_from	Stores supported daily-meal or monthly-plan prices.
operating_hours	hours_id, mess_id, day_of_week, opening_time, closing_time, is_closed	Stores operating schedule and closed days.

reviews	review_id, mess_id, student_id, rating, comment, status, created_at	Stores student feedback and moderation status.
admin_actions (optional)	action_id, admin_id, target_type, target_id, action, reason, created_at	Provides a history of important administrative actions.

6. Data Validation Rules

Required fields such as account email, mess name, address, and required pricing/menu fields must not be empty.

Email values must follow a valid email format; duplicate account emails should be rejected.

Passwords must be stored as a secure hash, never as plain text.

Ratings must be restricted to the chosen scale, recommended as 1 to 5.

Prices must be numeric and non-negative.

Latitude and longitude must fall within valid geographic ranges when coordinates are stored.

Foreign keys should ensure that a menu, price, or review refers to an existing mess and that owner/student references refer to valid accounts.

7. External Interface Requirements

7.1 User Interface

Provide separate student-facing pages, owner dashboard pages, and admin dashboard pages.

Forms should show field-level validation and understandable error messages.

Search results should display key comparison information such as name, distance when available, price, food type, rating, and availability.

The layout should be usable on common desktop and mobile screen sizes.

7.2 Software/API Interface

React.js communicates with the Spring Boot backend through HTTP REST APIs, typically exchanging JSON. The backend communicates with MySQL through a database access layer. The map provider is accessed through its supported API or embed mechanism.

Illustrative API route names (final routes are a design decision):

Method	Example route	Purpose
POST	/api/auth/register	Create an account.
POST	/api/auth/login	Authenticate a user.
GET	/api/messes	Search/list approved messes with optional filters.

GET	/api/messes/{id}	Get details for one mess.
POST	/api/owner/messes	Submit a mess listing.
PUT	/api/owner/messes/{id}	Update the owner's listing.
PUT	/api/admin/messes/{id}/status	Approve/reject or change listing status.
POST	/api/messes/{id}/reviews	Submit a review/rating.
GET	/api/messes/{id}/reviews	Get reviews for a mess.

These are examples for planning, not already implemented endpoints. The development team should finalize route names and request/response schemas.

8. Non-Functional Requirements

Category	Requirement
Security	Use secure password hashing, protected sessions/tokens, server-side role checks, input validation, and safe error messages.
Privacy	Request location permission only when needed; explain why it is used and provide a manual-location option where feasible.
Performance	Common pages and searches should respond promptly under the expected project demonstration load. The team should define and test a measurable target.
Reliability	The application should handle unavailable map services, invalid input, and database/API failures without crashing the user interface.
Usability	Use consistent navigation, readable labels, clear status messages, and responsive layouts.
Maintainability	Separate React UI, Spring Boot controllers/services/repositories, and database concerns so each layer can be updated independently.
Compatibility	Support current mainstream browsers used for the project demonstration.
Data integrity	Use database constraints and transactions where necessary to prevent inconsistent records.

Scalability	Keep search, data access, and API design structured so more mess listings and users can be supported later.

9. Security Requirements

All protected backend endpoints must require valid authentication.

Role-based authorization must be checked on the server for owner and admin operations.

Owners must not be able to modify another owner's mess listing by changing an ID in a request.

Passwords must be hashed using a modern password-hashing algorithm via a trusted library.

Validate and sanitize user input, and use parameterized database access to reduce injection risk.

Do not expose passwords, secret keys, tokens, or internal exception details in API responses or logs.

Use HTTPS in deployment environments that handle real user accounts.

10. Error and Exception Handling

Return an appropriate status and clear message for invalid input, unauthorized access, missing records, and server errors.

If location access is denied, allow the user to continue using a manual location or show a clear limitation.

If a map provider fails, retain the mess address and contact details when available.

If a save fails, tell the user that the change was not saved; do not display a false success message.

11. Testing and Acceptance Criteria

Test ID	Test	Expected result
TC-01	Register with valid data	Account is created and assigned the selected permitted role.
TC-02	Register with duplicate email	Registration is rejected with a clear message.
TC-03	Owner submits a mess	Listing is saved with Pending status and is not publicly visible.
TC-04	Admin approves listing	Listing status becomes Approved and it appears in student discovery if active.
TC-05	Admin rejects listing	Listing is not publicly visible and rejection status/reason is

		available to the owner if implemented.
TC-06	Student filters by price/food type	Results match the selected supported filters.
TC-07	Owner edits another owner's listing	Backend denies the operation.
TC-08	Student submits valid rating	Rating/review is saved and associated with the correct student and mess.
TC-09	Invalid price or rating	Backend rejects invalid data and returns a clear error.
TC-10	Location permission denied	Application handles denial gracefully and offers a manual-location route where implemented.

12. Constraints and Open Decisions

Select the map/location provider and confirm its pricing, API key handling, and usage limits.

Decide whether students must log in to search or only to submit reviews.

Decide whether one owner can manage multiple mess branches.

Define the required fields and evidence for mess verification.

Define whether reviews require proof of visiting/using a mess and whether one student may review a mess more than once.

Confirm whether email verification and email notifications are part of the first release.

Define measurable performance targets and expected concurrent users before formal load testing.

Define deployment environment, backups, and production data-retention rules if the application will be used beyond a college demonstration.

13. Glossary

Term	Simple meaning
BRD	Business Requirements Document: why the project is needed and what business outcomes it targets.
FRD	Functional Requirements Document: what each feature must do.

SRS	Software Requirements Specification: detailed software behavior, interfaces, data, security, and quality requirements.
REST API	A way for the frontend and backend to exchange requests and data over HTTP.
Frontend	The screens and interactions users see; in this project, React.js.
Backend	The server-side rules and processing; in this project, Java Spring Boot.
Database	Structured storage for accounts, messes, menus, prices, and reviews; in this project, MySQL.
Authentication	Checking who the user is, usually through login.
Authorization	Checking what an authenticated user is allowed to do.
CRUD	Create, Read, Update, Delete operations on data.
Validation	Checking whether submitted data is complete and correct.
Non-functional requirement	A quality or constraint such as security, usability, reliability, or performance.

Recommended Development Order

Design the database tables and relationships.

Implement registration, login, and server-side role authorization.

Implement owner mess submission and admin approval/rejection.

Implement student mess listing, search, filters, and detail pages.

Implement owner menu, price, timing, and availability management.

Integrate maps/location search.

Implement reviews and admin moderation.

Test access control, validation, error handling, and the complete end-to-end workflows.

MESSMAP

PROJECT DEVELOPMENT PHASES

Additional section for the Software Requirements Specification (SRS)

Purpose: This section defines the 12 planned phases for developing MessMap, from requirement analysis through deployment, documentation, and maintenance. It is added after the existing SRS without changing the original seven pages.

12-Phase Development Plan

Phase	Phase Name	Main Activities	Expected Deliverables
Phase 1	**Requirement Analysis**	Identify the problem statement, project objectives, stakeholders, user roles, functional and non-functional requirements, business rules, and project scope.	BRD, FRD, SRS, finalized requirements list
Phase 2	**System Planning**	Divide the project into modules, prioritize features, identify dependencies, allocate tasks among team members, estimate time, and prepare the development schedule.	Project plan, module breakdown, task allocation, project timeline
Phase 3	**System Design**	Design the overall system architecture, define frontend-backend communication, prepare student, owner, and admin workflows, and identify API requirements.	System architecture diagram, workflow diagrams, use-case diagram, API design outline
Phase 4	**Database Design**	Identify entities, define table attributes, establish primary and foreign keys, design relationships, normalize data where appropriate, and define validation constraints.	ER diagram, database schema, data dictionary, SQL table creation scripts
Phase 5	**UI/UX Design**	Design the home page, login and registration pages, student search interface, mess details page, owner dashboard, admin dashboard, forms, and navigation flow.	UI wireframes, page designs, navigation flowchart, reusable component plan
Phase 6	**Environment Setup**	Install and configure React.js, Java Spring Boot, and MySQL; create frontend and backend projects; configure database connectivity; and set up GitHub for collaboration.	Configured development environment, project folder structure, GitHub repository, initial application setup
Phase 7	**Backend Development**	Develop REST APIs, database operations, registration and login, role-based authorization, mess registration, admin verification, search and filtering, menu management, and review functionality.	Spring Boot backend, REST APIs, business logic, database integration, API testing results
Phase 8	**Frontend Development**	Develop React components and pages, implement registration and login forms, create student, owner, and admin interfaces, add form validation, and connect UI actions to backend APIs.	React.js frontend, responsive pages, user dashboards, search interface, frontend validation
Phase 9	**Module Integration**	Connect the React frontend, Spring Boot backend, MySQL database, and selected map service. Integrate authentication, mess registration, verification, search, owner updates, and reviews.	Integrated MessMap application, connected modules, end-to-end workflow results
Phase 10	**Testing and Quality Assurance**	Perform unit, API, integration, functional, and security testing. Verify role permissions, input validation, search results, error handling, and complete user workflows. Fix defects and perform regression testing.	Test plan, test cases, test report, defect report, corrected application
Phase 11	**Deployment**	Prepare the production build, configure hosting and environment variables, deploy the frontend and backend, configure the database and map service, and verify the deployed application.	Deployed MessMap website, application URL, configured backend and database, deployment guide
Phase 12	**Documentation and Maintenance**	Prepare the final project report, include diagrams and screenshots, document implementation and testing results, prepare the presentation, fix reported issues, and plan future improvements.	Final project report, presentation, user guide, maintenance plan, future enhancement list